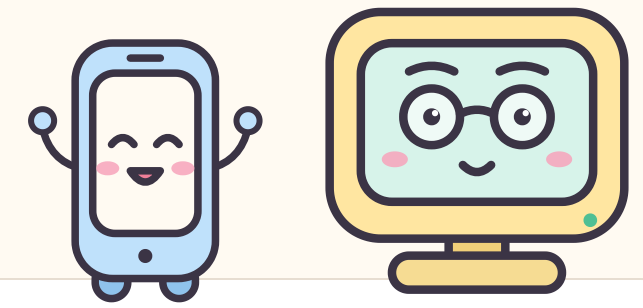


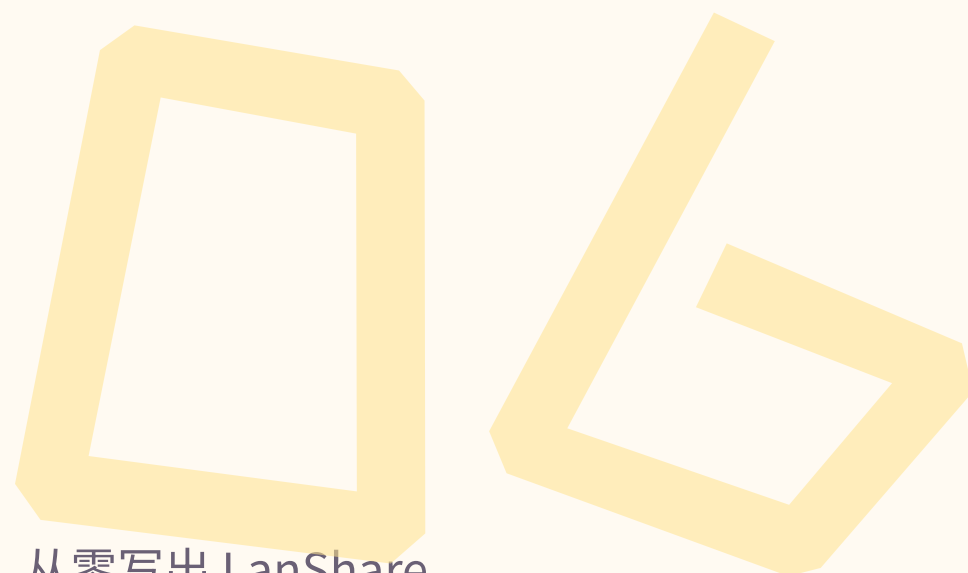


第 6 集 · 约 24 分钟

WASM①：把核心导出给 JavaScript

两种语言之间，只能传数字

配套阅读：书的第 3 章



从零写出 LanShare

这一集你会学到

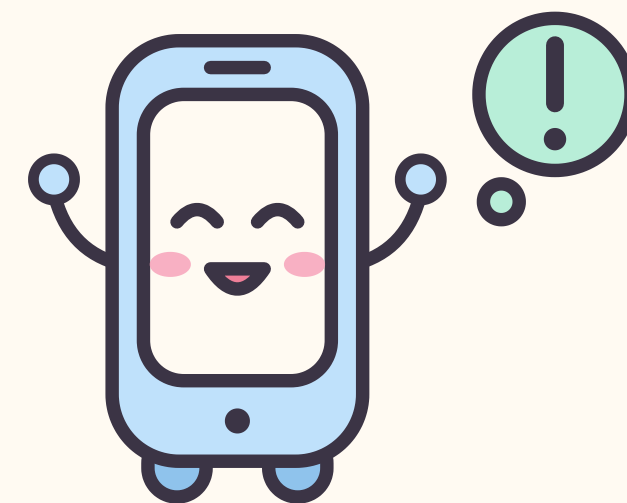
1. 线性内存：JS 和 WASM 怎么用“地址 + 长度”传一段字节
2. 裸指针和 unsafe：编译器管不到的地方，由人来担保
3. `#[unsafe(no_mangle)]` 和 `extern "C"`：导出函数、导入函数
4. `thread_local`、`RefCell`、`Option.take`：把配对状态存在 Rust 这边
5. 条件编译：随机数在浏览器里从 JS 借



先想一个问题

浏览器不是自带加密 API 吗？为什么还要自己把 Rust 编译进去？

小手机打开的是 http 加局域网 IP，不是浏览器眼里的“安全上下文”，自带的 crypto.subtle 根本不存在。





跨过边界，只能传数字





为什么不用 wasm-bindgen

- wasm-bindgen 能自动生成胶水代码，是最常用的工具
- 要另装 wasm-bindgen-cli，版本必须和库严格一致
- 我们只导出十几个函数，手写胶水一百多行
- 编出来的 lanshare.wasm 不到 100 KB



第一行：给协议核心起个短名字

lib.rs crates/lanshare-wasm/src/

第 1-11 行 • 共 11 行

```
1  //! 把 `lanshare-proto` 导出给浏览器：裸 C ABI，不依赖 wasm-bindgen。
2  //!
3  //! 调用约定（由 `web/proto.js` 遵守）：
4  //! - 所有指针都指向本模块线性内存里、由 JS 通过 [`ls_alloc`] 申请的缓冲区；
5  //! - 输入缓冲区长度由参数给出，输出缓冲区的长度由函数文档规定，JS 必须先申请够；
6  //! - 64 位计数器拆成高低两个 u32 传，兼容不支持 BigInt↔i64 的老浏览器。
7  //!
8  //! 浏览器里的 WASM 拿不到操作系统随机源，随机数通过导入函数 `env.ls_random`
9  //! 由 JS 的 `crypto.getRandomValues` 提供（它在 http 页面里也可用）。
10
11 use lanshare_proto as proto;
```



unsafe: 编译器管不到的地方

- 裸指针：只是一个地址，编译器不知道它后面有什么
- unsafe fn：调用我要小心，必须满足文档里写的条件
- 每个 unsafe 块上面写一行 SAFETY，说明为什么这里成立
- 本项目的规矩：unsafe 只出现在 WASM 边界和调用 Windows API 的地方



把地址变回切片

● ● ●

lib.rs

crates/lanshare-wasm/src/

第 20-39 行 • 共 39 行

```

20     }
21 }
22
23 /// SAFETY: 调用方保证 `ptr` 指向至少 N 字节的有效内存。
24 unsafe fn input_array<const N: usize>(ptr: *const u8) -> [u8; N] {
25     // SAFETY: 由本函数的调用约定保证; 长度正好是 N, try_into 不会失败。
26     unsafe { input(ptr, N) }.try_into().expect("长度正好是 N")
27 }
28
29 /// SAFETY: 调用方保证 `ptr` 指向至少 `data.len()` 字节的可写内存。
30 unsafe fn write_out(ptr: *mut u8, data: &[u8]) {
31     if !data.is_empty() {
32         // SAFETY: 由本函数的调用约定保证; 源与目标不重叠 (data 在 Rust 自己的分配里)。
33         unsafe { std::ptr::copy_nonoverlapping(data.as_ptr(), ptr, data.len()) };
34     }
35 }
36
37 fn counter(hi: u32, lo: u32) -> u64 {
38     (u64::from(hi) << 32) | u64::from(lo)
39 }

```



申请和释放内存

● ● ●

lib.rs

crates/lanshare-wasm/src/

第 39-58 行 • 共 58 行

```

39  }
40
41  /// 申请 `len` 字节，返回指针给 JS 写数据。用完必须用同样的 `len` 调 [`ls_free`].
42  #[unsafe(no_mangle)]
43  pub extern "C" fn ls_alloc(len: usize) -> *mut u8 {
44      let mut buf = Vec::<u8>::with_capacity(len.max(1));
45      let ptr = buf.as_mut_ptr();
46      std::mem::forget(buf);
47      ptr
48  }
49
50  /// 释放 [`ls_alloc`] 申请的内存。
51  ///
52  /// # Safety
53  /// `ptr` 和 `len` 必须正好来自一次 `ls_alloc(len)`，且只释放一次。
54  #[unsafe(no_mangle)]
55  pub unsafe extern "C" fn ls_free(ptr: *mut u8, len: usize) {
56      // SAFETY: ptr 来自 Vec::with_capacity(len.max(1))，长度按 0 重建即可，析构时只释放容量。
57      drop(unsafe { Vec::from_raw_parts(ptr, 0, len.max(1)) });
58  }

```



加密和解密

lib.rs

crates/lanshare-wasm/src/

第 90-109 行 • 共 109 行

```

90     key: *const u8,
91     ctr_hi: u32,
92     ctr_lo: u32,
93     aad: *const u8,
94     aad_len: usize,
95     sealed: *const u8,
96     sealed_len: usize,
97     out: *mut u8,
98 ) -> i32 {
99     // SAFETY: 由函数的调用约定保证。
100     let (key, aad, sealed) = unsafe { (input_array::<32>(key), input(aad, aad_len), input(sealed, sealed_len)) };
101     match proto::open(&key, counter(ctr_hi, ctr_lo), aad, sealed) {
102         Ok(plain) => {
103             // SAFETY: plain.len() == sealed_len - 16, 调用方已按此申请 out。
104             unsafe { write_out(out, &plain) };
105             0
106         }
107         Err(_) => 1,
108     }
109 }

```



握手的四个函数

lib.rs

crates/lanshare-wasm/src/

第 130-149 行 • 共 154 行

```

130
131 /// 口令配对的会话秘密，32 字节写到 `out`。
132 ///
133 /// # Safety
134 /// `pk` 有 `pk_len` 字节，`cid` 16 字节，`out` 32 字节可写。
135 #[unsafe(no_mangle)]
136 pub unsafe extern "C" fn ls_session_from_pake(pk: *const u8, pk_len: usize, cid: *const u8, out: *mut u8) {
137     // SAFETY: 由函数的调用约定保证。
138     unsafe { write_out(out, &proto::session_secret_from_pake(input(pk, pk_len), &input_array(cid))) };
139 }
140
141 /// 两把方向密钥：`out` 前 32 字节是 c2s，后 32 字节是 s2c。
142 ///
143 /// # Safety
144 /// `ss` 32 字节，`out` 64 字节可写。
145 #[unsafe(no_mangle)]
146 pub unsafe extern "C" fn ls_channel_keys(ss: *const u8, out: *mut u8) {
147     // SAFETY: 由函数的调用约定保证。
148     let keys = proto::ChannelKeys::derive(&unsafe { input_array(ss) });
149     // SAFETY: out 至少 64 字节。

```



确认标签

lib.rs

crates/lanshare-wasm/src/

第 175–194 行 • 共 195 行

```
175
176 /// 常量时间校验 PAKE 确认标签：通过返回 1，否则 0。
177 ///
178 /// # Safety
179 /// 各输入按长度有效，`cid` 16 字节。
180 #[unsafe(no_mangle)]
181 pub unsafe extern "C" fn ls_verify_confirm(
182     pk: *const u8,
183     pk_len: usize,
184     role: *const u8,
185     role_len: usize,
186     cid: *const u8,
187     tag: *const u8,
188     tag_len: usize,
189 ) -> i32 {
190     // SAFETY: 由函数的调用约定保证。
191     let (pk, role, cid, tag) =
192         unsafe { (input(pk, pk_len), input(role, role_len), input_array::<16>(cid), input(tag, tag_len)) };
193     let role = std::str::from_utf8(role).unwrap_or("");
194     i32::from(proto::verify_confirm(pk, role, &cid, tag))
```



配对状态放哪？

- PAKE 分两步：start 和 finish，中间状态要找地方存
- 存在 Rust 这边的一张全局表里，JS 只拿一个编号
- thread_local + RefCell：WASM 是单线程的，这就是全局变量
- 表里存 Option，finish 时 take 拿走，留下 None



口令配对的状态表

lib.rs crates/lanshare-wasm/src/

第 237-256 行 • 共 256 行

```
237 ///
238 /// # Safety
239 /// `peer` 按长度有效, `key_out` 32 字节可写。
240 #[unsafe(no_mangle)]
241 pub unsafe extern "C" fn ls_pake_finish(handle: u32, peer: *const u8, peer_len: usize, key_out: *mut u8) -> i32 {
242     let state = PAKE.with_borrow_mut(|table| {
243         let index = (handle as usize).checked_sub(1)?;
244         table.get_mut(index)?.take()
245     });
246     let Some(state) = state else { return 1 };
247     // SAFETY: 由函数的调用约定保证。
248     match proto::pake_finish(state, unsafe { input(peer, peer_len) }) {
249         Ok(key) => {
250             // SAFETY: key_out 32 字节可写。
251             unsafe { write_out(key_out, &key) };
252             0
253         }
254         Err(_) => 1,
255     }
256 }
```



浏览器里的随机数从哪来



- 浏览器里的 WASM 没有操作系统，拿不到系统随机源
- 声明一个导入函数 `ls_random`，由 JS 实现
- JS 用 `crypto.getRandomValues`，它在 http 页面里也能用
- 告诉 `getrandom`：要随机数时调我们的函数



依赖：只在 WASM 上生效的那一个



Cargo.toml

crates/lanshare-wasm/

第 1-16 行 · 共 16 行

```
1 [package]
2 name = "lanshare-wasm"
3 description = "把 lanshare-proto 导出给浏览器（裸 C ABI，不依赖 wasm-bindgen）"
4 version.workspace = true
5 edition.workspace = true
6 publish.workspace = true
7
8 [lib]
9 crate-type = ["cdylib", "rlib"]
10
11 [dependencies]
12 lanshare-proto = { path = "../lanshare-proto" }
13 rand_core = { version = "0.6", features = ["getrandom"] }
14
15 [target.'cfg(target_arch = "wasm32")'.dependencies]
16 getrandom = { version = "0.2", features = ["custom"] }
```



从 JS 借随机数

lib.rs crates/lanshare-wasm/src/

第 14-33 行 • 共 273 行

```
14
15 #[cfg(target_arch = "wasm32")]
16 #[link(wasm_import_module = "env")]
17 unsafe extern "C" {
18     /// 由 JS 提供：往 `[ptr, ptr+len)` 写入密码学安全的随机字节。
19     fn ls_random(ptr: *mut u8, len: usize);
20 }
21
22 #[cfg(target_arch = "wasm32")]
23 fn js_random(buf: &mut [u8]) -> Result<(), getrandom::Error> {
24     // SAFETY: buf 是一段有效、可写、长度为 buf.len() 的线性内存；JS 端只在这个范围内写入。
25     unsafe { ls_random(buf.as_mut_ptr(), buf.len()) };
26     Ok(())
27 }
28
29 #[cfg(target_arch = "wasm32")]
30 getrandom::register_custom_getrandom!(js_random);
31
32 /// SAFETY: 调用方保证 `ptr` 指向至少 `len` 字节的有效内存（`len == 0` 时可以是任意值）。
33 unsafe fn input<'a>(ptr: *const u8, len: usize) -> &'a [u8] {
```



编译成 WASM



Windows PowerShell

```
PS D:\code\lanshare> rustup target add wasm32-unknown-unknown
info: component rust-std for target wasm32-unknown-unknown is up to date

PS D:\code\lanshare> cargo build -p lanshare-wasm --release --target wasm32-unknown-unknown
Compiling lanshare-proto v2.0.0 (D:\code\lanshare\crates\lanshare-proto)
Compiling lanshare-wasm v2.0.0 (D:\code\lanshare\crates\lanshare-wasm)
Finished `release` profile [optimized] target(s) in 1.21s

PS D:\code\lanshare> (Get-Item target\wasm32-unknown-unknown\release\lanshare_wasm.wasm).Length
95754
```





踩坑：导入函数要写明模块

```
Windows PowerShell

PS D:\code\lanshare> cargo build -p lanshare-wasm --release --target wasm32-unknown-unknown
   Compiling lanshare-wasm v2.0.0 (D:\code\lanshare\crates\lanshare-wasm)
error: linking with `rust-lld` failed: exit code: 1
|
= note: "rust-lld" "-flavor" "wasm" "--export" "__getrandom_custom" "--export" "ls_alloc" ..... (这一行很长, 省略: 全部链接参数)
= note: some arguments are omitted. use `--verbose` to show all linker arguments
= note: rust-lld: error: D:\code\lanshare\target\wasm32-unknown-unknown\release\deps\lanshare_wasm.lanshare_wasm.5fbf205d404e124-cgu.0.rcgu.o: undefined symbol: ls_random

error: could not compile `lanshare-wasm` (lib) due to 1 previous error
```



选择题

ls_alloc 里如果去掉 `std::mem::forget(buf)`，会怎样？

- A 没影响，只是少了一行
- B 返回的地址指向已经释放的内存，JS 往里写会破坏 Rust 的堆
- C 编译不通过



选择题

ls_alloc 里如果去掉 `std::mem::forget(buf)`，会怎样？

A 没影响，只是少了一行

少了这一行，buf 在函数返回时就被释放了。

C 编译不通过

这段代码照样能编译，这正是它危险的地方。

B 返回的地址指向已经释放的内存，JS 往里写会破坏 Rust 的堆

不调用 forget，buf 在函数结束时就被释放，JS 拿到的是悬空的地址，往里写数据就会写坏别的东西。



选择题

同一个 PAKE 句柄，第二次调用 `ls_pake_finish` 会返回什么？

A 0，成功

B 1，失败

C 程序崩溃



选择题

同一个 PAKE 句柄，第二次调用 `ls_pake_finish` 会返回什么？

A 0，成功

第一次 finish 时状态已经被 take 拿走了。

C 程序崩溃

`get_mut` 和 `take` 都不会崩溃，拿不到就是 `None`。

B 1，失败

第一次 take 之后那一格是 `None`，第二次拿到 `None`，直接返回 1。



选择题

计数器为什么拆成高低两个 u32 传给 WASM?

- A 因为 Rust 不支持 u64
- B 为了兼容不支持在 JS 和 WASM 之间直接传 64 位整数的老浏览器
- C 为了让加密更快



选择题

计数器为什么拆成高低两个 u32 传给 WASM?

A 因为 Rust 不支持 u64

Rust 当然支持 u64，计数器在 Rust 里就是 u64。

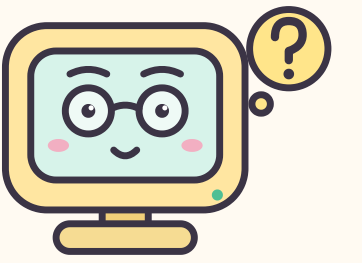
C 为了让加密更快

和速度无关，是兼容性的考虑。

B 为了兼容不支持在 JS 和 WASM 之间直接传 64 位整数的老浏览器

老版本的手机浏览器不支持这种传法，拆成两个 32 位最稳妥，Rust 那边再拼回来。

想一想



**这个文件里 `unsafe` 出现了很多次，可加密本身的代码里一个都没有。
为什么要把 `unsafe` 都挤在 `input`、`write_out` 这些小函数和边界上？**



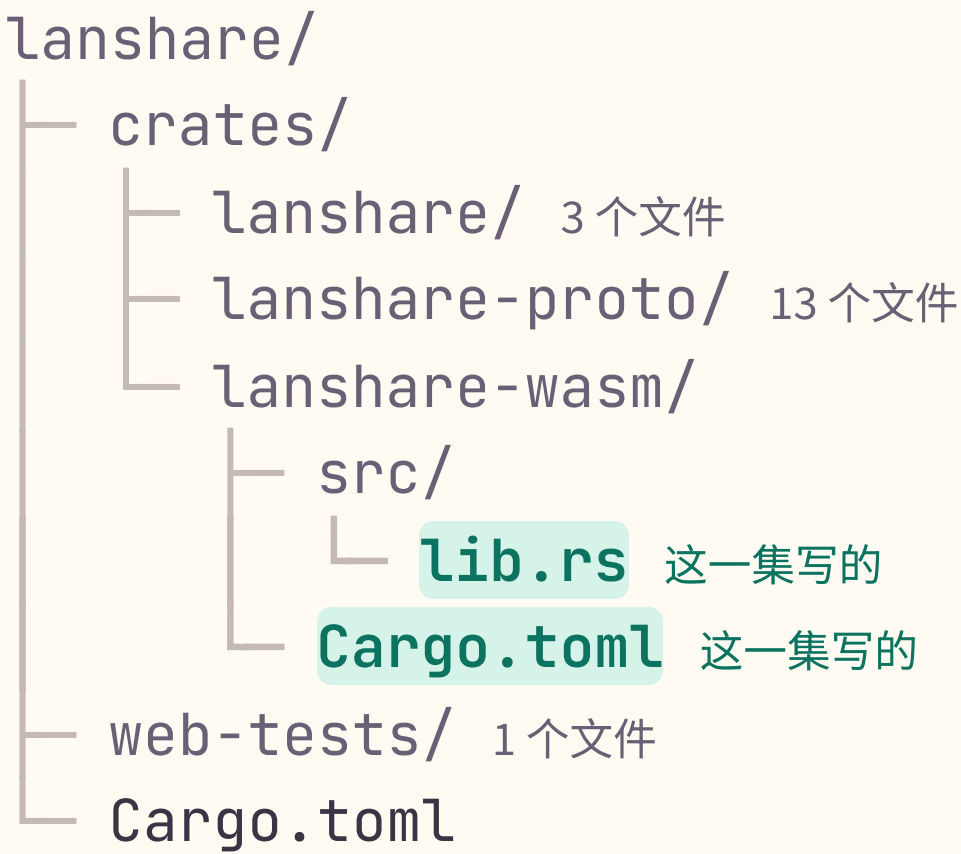
想一想

**这个文件里 `unsafe` 出现了很多次，可加密本身的代码里一个都没有。
为什么要把 `unsafe` 都挤在 `input`、`write_out` 这些小函数和边界上？**

`unsafe` 的意思是“编译器管不到，由人来担保”。担保的地方越少、越集中，越容易审查。这个文件里，所有危险操作都落在几个小工具上，每处写着 SAFETY 理由；真正的加密、握手、配对逻辑全在 `lanshare-proto` 里，是编译器能完全检查的安全 Rust。审代码的人只需要盯住边界：地址和长度是不是对的，申请和释放是不是一一对应。



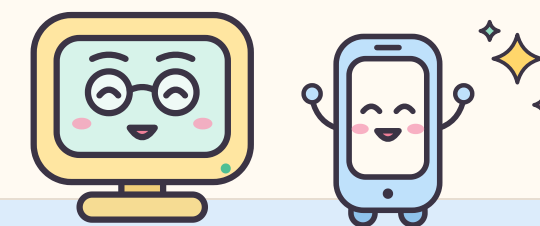
这一集写完了





这一集写了什么

- ✓ 跨过 JS 和 WASM 的边界只能传数字：字节靠“申请内存 + 地址 + 长度”
- ✓ unsafe 只出现在边界上，每处写 SAFETY 理由
- ✓ ls_alloc 用 forget 把内存交给 JS，ls_free 用 from_raw_parts 收回
- ✓ PAKE 状态存在 thread_local 的表里，Option.take 保证只能用一次
- ✓ 条件编译：只在 WASM 上从 JS 导入随机数



下一集

全课完结，感谢收看